

AI-Assisted Coding and Collaborative Project Development

Lecture 1: Kickoff

Alexis Shakas¹

TA: Ian Wasser²

¹Department of Earth and Planetary Science

²Department of Computer Science

February 17, 2026

Outline

Introduction

Research Oriented Programming

Your project

The command line interface

Summary

Outline

Introduction

Research Oriented Programming

Your project

The command line interface

Summary

What this course is about

This course has been developed as part of an [Innovedum grant](#) which provides funding to advance education and teaching at ETH.

The learning objectives of the course are:

What this course is about

This course has been developed as part of an [Innovedum grant](#) which provides funding to advance education and teaching at ETH.

The learning objectives of the course are:

1. Become a solid research programmer in Python following "best" practises and community standards

What this course is about

This course has been developed as part of an [Innovedum grant](#) which provides funding to advance education and teaching at ETH.

The learning objectives of the course are:

1. Become a solid research programmer in Python following "best" practises and community standards
2. Boost your coding skills with AI assistance while remaining in control and learning along the way

What this course is about

This course has been developed as part of an [Innovedum grant](#) which provides funding to advance education and teaching at ETH.

The learning objectives of the course are:

1. Become a solid research programmer in Python following "best" practises and community standards
2. Boost your coding skills with AI assistance while remaining in control and learning along the way
3. Learn to collaborate and version control your work through Git

What this course is about

This course has been developed as part of an [Innovedum grant](#) which provides funding to advance education and teaching at ETH.

The learning objectives of the course are:

1. Become a solid research programmer in Python following "best" practises and community standards
2. Boost your coding skills with AI assistance while remaining in control and learning along the way
3. Learn to collaborate and version control your work through Git
4. Create and distribute your Python project through Git

How do we get there

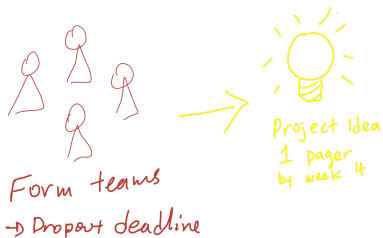


Form teams

→ Drop out deadline

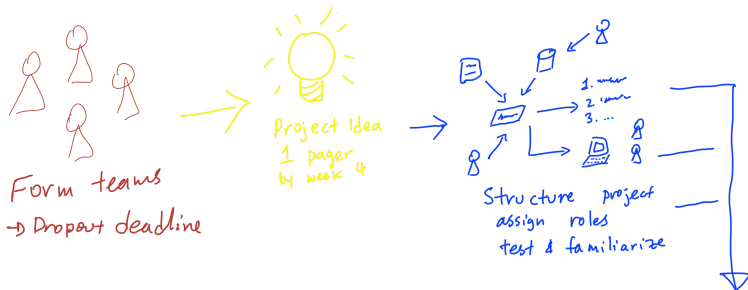
You will first team up and lay out a plan on how to collaborate

How do we get there



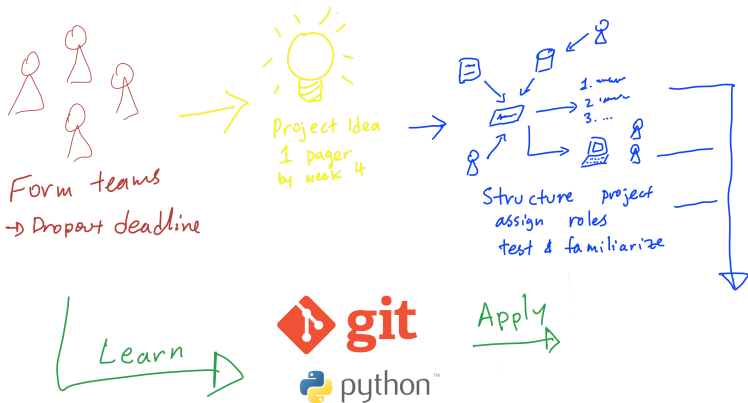
You will then define and elaborate on your project idea

How do we get there



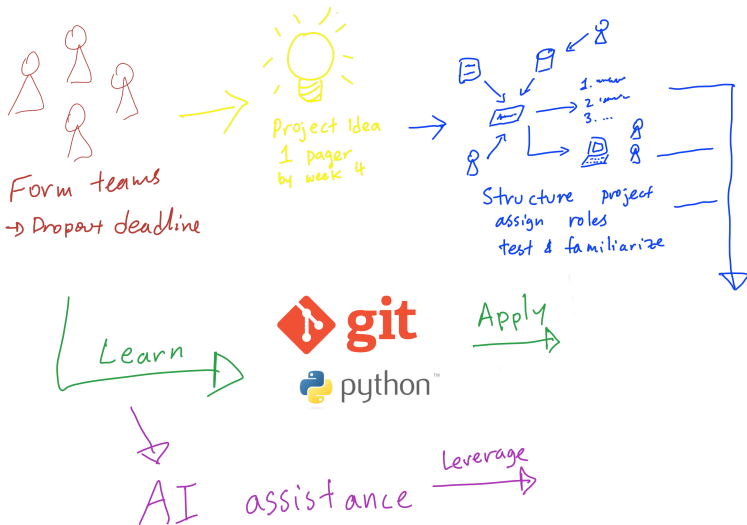
In the meantime, you'll be learning Git and Python

How do we get there



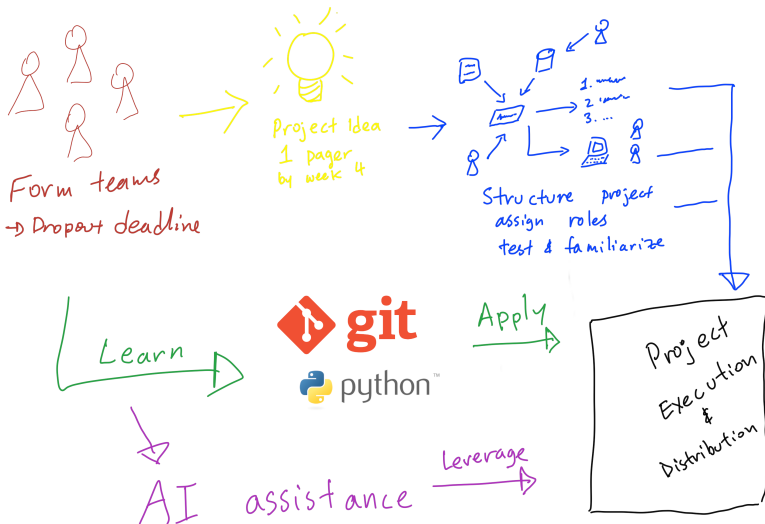
You'll learn Git and Python..

How do we get there



and leverage AI tools to achieve more, but with a focus on learning!

How do we get there



The project execution and distribution takes place only in the last few weeks!

Lecture dates

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	TBA
6: AI-assisted Python	24/3	Y	RZ D8
7: The 3 R's in Python	31/3	Y	TBA
8: Python packaging ecosystem	14/4	Y	TBA
9: Git advanced	21/4	N	RZ D8
10: OOP	28/4	Y	RZ D8
11: Testing and debugging	5/5	Y	TBA
12: Joker lecture	12/5	N	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A: Take home assignment

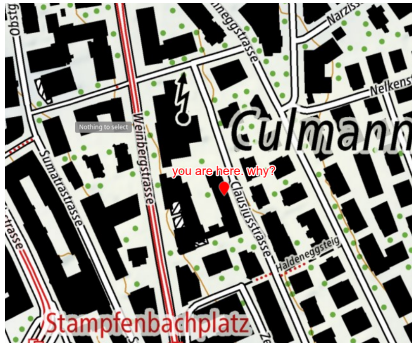
TBA: Lecture room to be announced

Today's learning objectives

In today's lecture, we'll focus on the following:

1. Learning and teaching philosophy
2. How to structure project ideas
3. The command line interface

Why are you here?



Instructor: Alexis Shakas

- ▶ 2011: BSc in Physics
- ▶ 2013: MSc in Applied Geophysics
- ▶ 2017: PhD in Geophysics
- ▶ 2018: Post-doc in Machine Learning
- ▶ 2019-2023: Post-doc in Bedretto Lab



Since 2024 I am scientific lead for GeoHEAT, a project aimed at advancing geothermal exploration. I coordinate forest projects in Switzerland and abroad. I "converted" to Python in 2018!

I am here because I really enjoy teaching, and I want to give you the opportunity to learn and create something in a dynamic learning environment.

Teaching Assistant: Ian Wasser

- ▶ Since 2023: BSc in Computer Science, ETH Zurich
- ▶ Since 2024: Leading courses about Git as a board member of [the Alternative](#).



I am looking forward helping you understand all about programming and handling software projects. Ask me anything at anytime and I will try supporting you with your problems.

Teaching philosophy

- ▶ **This is a learning environment..for all of us**
Especially with AI tools. GenAI makes the classroom an interactive (and uncertain) environment for teachers. We're in this together!

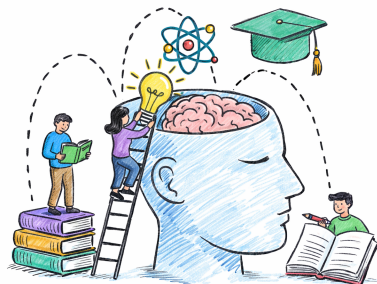
Teaching philosophy

- ▶ **This is a learning environment..for all of us**
Especially with AI tools. GenAI makes the classroom an interactive (and uncertain) environment for teachers. We're in this together!
- ▶ **AI tools and technologies are advancing fast and state of the art is hard to define**
The lecture material will evolve during the semester and your feedback will be essential to redefine and guide the course.

Teaching philosophy

- ▶ **This is a learning environment..for all of us**
Especially with AI tools. GenAI makes the classroom an interactive (and uncertain) environment for teachers. We're in this together!
- ▶ **AI tools and technologies are advancing fast and state of the art is hard to define**
The lecture material will evolve during the semester and your feedback will be essential to redefine and guide the course.
- ▶ **AI allows for personalized education.**
I am interested in how we can use AI and enhance learning at the same time. You'll have to find the best way to optimize your experience; and I am interested to hear about it!

How do you learn best?



Learning philosophy

- ▶ **Focus on the high-level approach, not the specific tool.**
There are endless options available so instead of learning specific tools, we'll focus on the high-level skills to apply them well. You can choose the tools that fit you best!

Learning philosophy

- ▶ **Focus on the high-level approach, not the specific tool.**
There are endless options available so instead of learning specific tools, we'll focus on the high-level skills to apply them well. You can choose the tools that fit you best!
- ▶ **(especially) With Python, there is no such concept as "mastering" the language.**
Instead, strive to apply your domain expertise to create robust scientific work. You will learn Python by solving real problems, not by lengthy recitals of complex syntax.

Learning philosophy

- ▶ **Focus on the high-level approach, not the specific tool.**
There are endless options available so instead of learning specific tools, we'll focus on the high-level skills to apply them well. You can choose the tools that fit you best!
- ▶ **(especially) With Python, there is no such concept as "mastering" the language.**
Instead, strive to apply your domain expertise to create robust scientific work. You will learn Python by solving real problems, not by lengthy recitals of complex syntax.
- ▶ **Question the outcome.**
It's easier than ever to generate code; this makes your role as a research developer even more important. You need to be critical, creative and alert!

Performance assessment

Your course grade will consist of two parts:

1. 5 take-home assignments, 8% each for 40% of your grade.
For each assignment you need to do **all** 3 tasks for the grade:
 - ▶ Turn it in
 - ▶ Reflect / provide self-assessment
 - ▶ Write peer review to a random classmate

Performance assessment

Your course grade will consist of two parts:

1. 5 take-home assignments, 8% each for 40% of your grade.
For each assignment you need to do **all** 3 tasks for the grade:
 - ▶ Turn it in
 - ▶ Reflect / provide self-assessment
 - ▶ Write peer review to a random classmate
2. Project, 60% of final grade
 - ▶ Group of 4-5
 - ▶ Use of Python and Git
 - ▶ Common grade for each team
 - ▶ Turn it in May 29th

Course Structure

The lecture will mostly follow this format:

- ▶ Lecture for 45 min, 10 min break, lecture for 50 min
- ▶ Lectures will be interactive with about 10 - 15 min of slides followed by practise sessions.
- ▶ In-class coding takes place so bring **charged** computers.
- ▶ You can collaborate with your neighbor and team

Most lectures will take place here (PBLabs) unless otherwise noted. The other lectures will (likely) take place at IFW, next door. Exact location will be announced on week 3.

Lecture streaming and recordings

Lecture will be streamed online via the following

▶ [zoom link](#)

The zoom room is password protected and the password is **kicode**.

In-class participation is expected! Lectures will not be recorded.

Course requirements

- ▶ Respect your classmates and teachers
- ▶ **Ask** questions during class!
- ▶ Know some Python or have strong willingness to learn
- ▶ Use AI responsibly (more next week)
- ▶ Have a personal computer ([Bring Your Own Device](#))
- ▶ Devote some time each week to learn

Lecture material

Our main course content exchange will happen on Moodle, although your project will live on Git (/gitlab, github, bitbucket..).

- ▶ Discussion forum: Anything class related, including questions, project ideas, collaboration requests, ..
- ▶ Resources and Guides: Post interesting things here with a short explanation.
- ▶ Lecture content (slides, exercises, datasets) will show up weekly, sometimes right before the lecture.

 **General**

 [Discussion forum](#)

 [Resources and Guides](#)

 [Virtual classroom](#)
Starts: Tuesday, 17 February 2026, 1:15 PM

 **Introduction** Current week

Code blocks

Whenever you see the block below in the lecture slides, this means that code is presented.

```
"""Some code will be shown here which you can basically copy
and paste into a Python command line"""
from my_fav_package import cool_stuff
print("Hello, world")
# do some other cool things
import antigravity
```

Code, especially values, variables, functions and classes may also be displayed in-line with a different font:

- ▶ `my_variable` is a python variable name
- ▶ `my_function` is a python function name

In-class exercises / participation

You will have several occasions to try out things during class. Many tasks will have the instructions directly on slides. Sometimes, you'll have to:

1. Download the exercise from Moodle
2. Run the code and interact either on your PC or online.

```
# %% CODE BLOCK STARTS  
code here  
# %% CODE BLOCK ENDS
```

3. Discuss with your neighbors
4. Go over the solutions together

Hidden slides

Some slides, like this one, will be "hidden" to you before the lecture. The aim is to avoid spoilers and keep the element of surprise. After each lecture, I will add the long version of the slide deck on Moodle (but they will mostly be unnecessary slides, like this one).



Figure made with AI.

Any questions?



Outline

Introduction

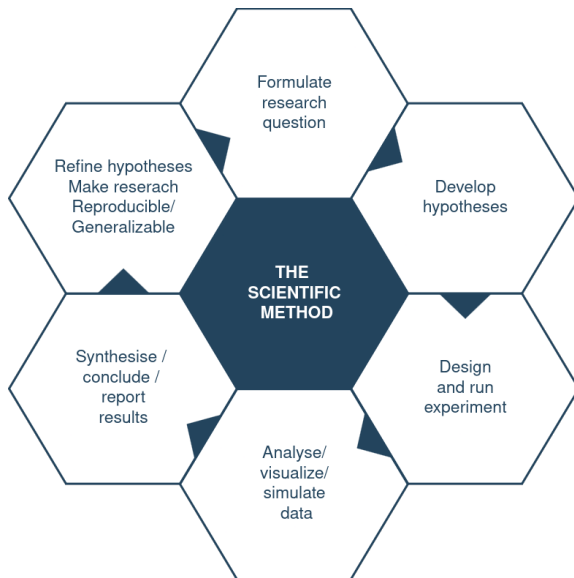
Research Oriented Programming

Your project

The command line interface

Summary

The (re)search for scientific knowledge



The (re)search for scientific knowledge



Research Oriented Programming

Research Oriented Programming refers to coding practices specifically tailored to support scientific research or academic inquiries:

- ▶ Solve complex problems, validate hypotheses.

Research Oriented Programming

Research Oriented Programming refers to coding practices specifically tailored to support scientific research or academic inquiries:

- ▶ Solve complex problems, validate hypotheses.
- ▶ Simulate natural phenomena or systems (e.g., climate models, biological systems, earthquakes, ...).

Research Oriented Programming

Research Oriented Programming refers to coding practices specifically tailored to support scientific research or academic inquiries:

- ▶ Solve complex problems, validate hypotheses.
- ▶ Simulate natural phenomena or systems (e.g., climate models, biological systems, earthquakes, ...).
- ▶ Analyze data collected from experiments or observations.

Research Oriented Programming

Research Oriented Programming refers to coding practices specifically tailored to support scientific research or academic inquiries:

- ▶ Solve complex problems, validate hypotheses.
- ▶ Simulate natural phenomena or systems (e.g., climate models, biological systems, earthquakes, ...).
- ▶ Analyze data collected from experiments or observations.
- ▶ Develop new algorithms, models, or methodologies

Research Oriented Programming

Research Oriented Programming refers to coding practices specifically tailored to support scientific research or academic inquiries:

- ▶ Solve complex problems, validate hypotheses.
- ▶ Simulate natural phenomena or systems (e.g., climate models, biological systems, earthquakes, ...).
- ▶ Analyze data collected from experiments or observations.
- ▶ Develop new algorithms, models, or methodologies

Research programmers should strive to create **reproducible** workflows and **readable** code that facilitates **collaboration** and **re-usability**.



Research programmer, someone who applies programming skills and domain expertise to support and advance their research activities (figure made with AI).

Focus on good practices

Research Oriented Programmers should strive for software that is:

- ▶ Reproducible: obtaining consistent results using the same input data, computational steps, methods, and code. Tests will play a major role here!

Focus on good practices

Research Oriented Programmers should strive for software that is:

- ▶ Reproducible: obtaining consistent results using the same input data, computational steps, methods, and code. Tests will play a major role here!
- ▶ Readable: "Code is read much more than it is written" - Guido van Rossum.

Focus on good practices

Research Oriented Programmers should strive for software that is:

- ▶ Reproducible: obtaining consistent results using the same input data, computational steps, methods, and code. Tests will play a major role here!
- ▶ Readable: "Code is read much more than it is written" - Guido van Rossum.
- ▶ Reusable: foster use in multiple projects or contexts without modification. It is modular, maintainable, and scalable.

Your (current) level in Python



Outline

Introduction

Research Oriented Programming

Your project

The command line interface

Summary

The project

The course will be structured around a project that you will carry out as teams. The teams will be assigned by week 3. The deadline to choose or propose a project is week 4. You have two options:

The project

The course will be structured around a project that you will carry out as teams. The teams will be assigned by week 3. The deadline to choose or propose a project is week 4. You have two options:

1. Develop new project ideas, elaborated and refined with the team and lecturer, that fulfill certain criteria. Likely little time to advance on the research front.

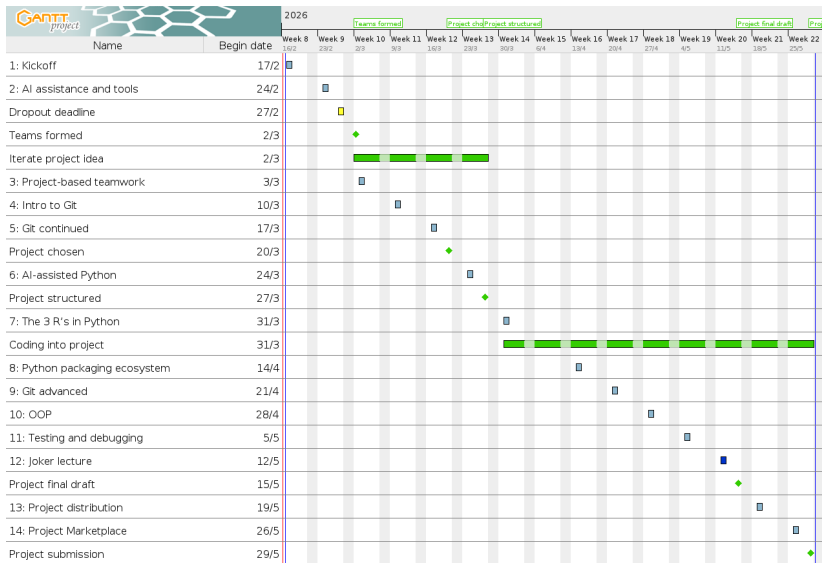
The project

The course will be structured around a project that you will carry out as teams. The teams will be assigned by week 3. The deadline to choose or propose a project is week 4. You have two options:

1. Develop new project ideas, elaborated and refined with the team and lecturer, that fulfill certain criteria. Likely little time to advance on the research front.
2. Choose from existing project ideas with some content and data available. Usually products of past research that need structure and can also be advanced scientifically by active collaboration with lead researcher. Earth-science related, currently 3-4 available.

Roadmap

Here is the project roadmap for the semester:



Project plan

By week-4, you and your team will have a **2-page** project plan that will outline your:

1. Mission
2. Scope
3. Objectives and success criteria
4. Inputs/Outputs
5. Constraints
6. Risks & mitigation strategies

1) Mission

A short description of one or a few sentences, structured similar to:

*We will build **X** for **Y** to enable **Z** under constraints **C**.*

1) Mission

A short description of one or a few sentences, structured similar to:

*We will build **X** for **Y** to enable **Z** under constraints **C**.*

Here are some examples from last year:

- ▶ A 2D seismic wave simulator for researchers to explore heterogeneity effects on travel times running on a laptop.
- ▶ A dashboard where users can drop in datasets and perform exploratory data analysis and visualizations.
- ▶ An online, multi-user version of the game "Brändi Dog".
- ▶ An orbital animation GUI where users can define and simulate solar systems in 2D.

2) Scope

You will outline the scope, the "what" and "how much" of the project:

- ▶ **In scope** (3–5 bullets)
- ▶ **Out of scope / non-goals** (at least 3 bullets)

Non-goal examples:

- ▶ No distributed computing
- ▶ No GPU acceleration until baseline is correct
- ▶ Not a Graphical User Interface

3) Objectives

You will define SMART objectives; Specific, Measurable, Achievable, Relevant, and Time-bound.

The objectives will directly address your projects'

Scientific validity

May include analytic benchmarking, error tolerance, application on known datasets, reproducibility, ...

Operational performance

Runtime/memory targets, readability and reusability, multi-platform operability..

4) Inputs/Outputs

Early on, you will specify (not a full list):

- ▶ **Data formats:** For example, NetCDF, GeoTIFF, HDF5, CSV, shapefiles, ...
- ▶ **Scale:** image size, grid size, timesteps, file sizes, ...
- ▶ **Units/coordinates:** meters vs degrees, reference frames, time bases, ...
- ▶ **Metadata:** rasters/vectors, plots, reports, logs, ... or anything produced as outcome of the project.

While this may change along the way, it will drive the boundaries of your project and focus your work.

5) Constraints

You will come up with constraints for your project, including, but not limited to:

- ▶ **Compute:** CPU/GPU, RAM limits, ...
- ▶ **Runtime:** interactive vs batch
- ▶ **Dependencies:** libraries, licensing, ...
- ▶ **Data governance:** sensitive/embargoed data
- ▶ **Platform:** Windows, mac, Linux, ...

6) Risks & mitigation strategies

You will come up with potential risks and possible mitigation plans.

Risk	Mitigation Plan
Data Inconsistencies	Implement a Data Validation Layer using filtering or preprocessing scripts.
Numerical Stability	Use Assertion Testing and boundary tests to evaluate numerical overflows.
Library Incompatibility	Utilize Environment Encapsulation (e.g., Docker, Conda, or <code>requirements.txt</code>) to lock dependency versions and ensure portability across systems.

Your (current) level in Git



git



Outline

Introduction

Research Oriented Programming

Your project

The command line interface

Summary

What is it?

The command line interface (often called the terminal, console, shell, ..) is a **user interface** that allows to interact with the computer through text input.

- ▶ On Windows: Cmd.exe, Windows power shell, Command Prompt (Git BASH)
- ▶ On Linux and MacOS: Terminal, zsh, bash, ..

What can you do with it?

There are many things you can do with the command line. Here is a non-exhaustive list:

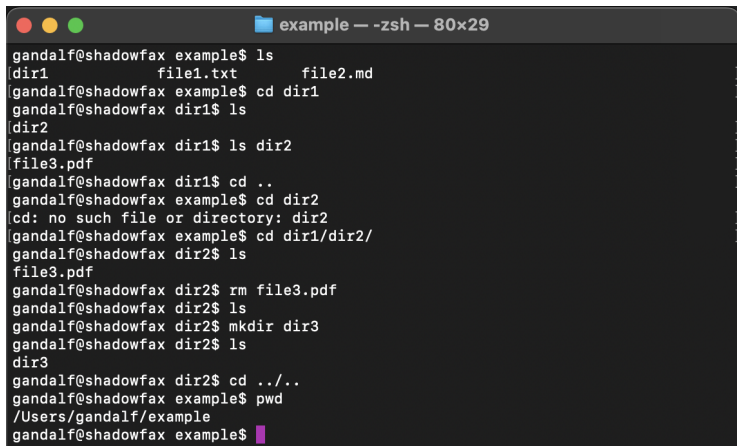
- ▶ Navigate through files and folders
- ▶ Perform file operations
- ▶ Retrieve system information
- ▶ Edit / compress / transfer files
- ▶ Interact with remote machines
- ▶ Automate tasks

And most importantly...



using the terminal looks cool!

Command Line Example MacOS



```
example — zsh — 80x29
gandalf@shadowfax example$ ls
[dir1          file1.txt          file2.md      ]
gandalf@shadowfax example$ cd dir1
gandalf@shadowfax dir1$ ls
[dir2          ]
gandalf@shadowfax dir1$ ls dir2
[file3.pdf     ]
gandalf@shadowfax dir1$ cd ..
gandalf@shadowfax example$ cd dir2
[cd: no such file or directory: dir2]
gandalf@shadowfax example$ cd dir1/dir2/
gandalf@shadowfax dir2$ ls
[file3.pdf     ]
gandalf@shadowfax dir2$ rm file3.pdf
gandalf@shadowfax dir2$ ls
gandalf@shadowfax dir2$ mkdir dir3
gandalf@shadowfax dir2$ ls
[dir3          ]
gandalf@shadowfax dir2$ cd ../../
gandalf@shadowfax example$ pwd
/Users/gandalf/example
gandalf@shadowfax example$
```

Figure: Example use of command line in MacOS terminal

Navigating your computer

`pwd`: Print Working Directory shows the complete path to the current directory

Navigating your computer

`pwd`: Print Working Directory shows the complete path to the current directory

`cd`: Change directory

- ▶ `cd` move to your home directory
- ▶ `cd mydir` move to directory `mydir` (try `cd /`)
- ▶ `cd ..` move to the previous (parent) directory

Navigating your computer

`pwd`: Print Working Directory shows the complete path to the current directory

`cd`: Change directory

- ▶ `cd` move to your home directory
- ▶ `cd mydir` move to directory `mydir` (try `cd /`)
- ▶ `cd ..` move to the previous (parent) directory

`mkdir`: create a new directory

- ▶ `mkdir mydir` create a new directory named `mydir` within the current directory
- ▶ `mkdir /mydir` create a new directory named `mydir` within the root directory

Navigating your computer

`pwd`: Print Working Directory shows the complete path to the current directory

`cd`: Change directory

- ▶ `cd` move to your home directory
- ▶ `cd mydir` move to directory `mydir` (try `cd /`)
- ▶ `cd ..` move to the previous (parent) directory

`mkdir`: create a new directory

- ▶ `mkdir mydir` create a new directory named `mydir` within the current directory
- ▶ `mkdir /mydir` create a new directory named `mydir` within the root directory

`ls`: List contents of a directory

- ▶ `ls` list contents of current directory
- ▶ `ls mydir` list contents of `mydir`

Important note on paths

In Windows, folders are separated by \ while in MacOS and Linux they are separated by /.

Important note on paths

In Windows, folders are separated by \ while in MacOS and Linux they are separated by /.

Definition (Relative Path)

A *relative* path is always resolved *relative* to the current working directory (`pwd`) and never starts with a / or '\ ' of character

Important note on paths

In Windows, folders are separated by \ while in MacOS and Linux they are separated by /.

Definition (Relative Path)

A *relative* path is always resolved *relative* to the current working directory (`pwd`) and never starts with a / or '\ ' of character

Definition (Absolute Path)

An *absolute* path always starts at the root directory and includes the full path to a directory or file.

Important note on paths

In Windows, folders are separated by `\` while in MacOS and Linux they are separated by `/`.

Definition (Relative Path)

A *relative* path is always resolved *relative* to the current working directory (`pwd`) and never starts with a `/` or `'\'` of character

Definition (Absolute Path)

An *absolute* path always starts at the root directory and includes the full path to a directory or file.

- ▶ When working with the command line, file or directory locations can always be given as either relative paths or absolute paths, e.g., `cd project` or `cd /Users/myuser/coding/project`

MiniTask

Open the CLI and make a new directory

1. Find the terminal/CLI on your computer and open it
2. List the path contents of the directory which opens by default (usually home directory)
3. Create a new directory called `temp_dir` and `cd` into it

File operations

`touch`: Create or change modification time of a file

- ▶ `touch newfile`: Create a file within the current directory called `newfile` assuming it does not exist
- ▶ `touch file`: Change the modification time to current

File operations

`touch`: Create or change modification time of a file

- ▶ `touch newfile`: Create a file within the current directory called `newfile` assuming it does not exist
- ▶ `touch file`: Change the modification time to current

`mv` and `cp`: Move or copy files between directories

- ▶ `mv old_directory/file new_directory/.` Move file from old to new directory; the `'.'` in the end is needed!
- ▶ `cp old_directory/file new_directory/new_filename`
Copy file from old to new directory and change name.
- ▶ Renaming and `'.'` works with both `mv` and `cp`.

File operations

`touch`: Create or change modification time of a file

- ▶ `touch newfile`: Create a file within the current directory called `newfile` assuming it does not exist
- ▶ `touch file`: Change the modification time to current

`mv` and `cp`: Move or copy files between directories

- ▶ `mv old_directory/file new_directory/.` Move file from old to new directory; the `'.'` in the end is needed!
- ▶ `cp old_directory/file new_directory/new_filename`
Copy file from old to new directory and change name.
- ▶ Renaming and `'.'` works with both `mv` and `cp`.

`rm`: Remove files or directories

- ▶ `rm myfile`: Remove file `myfile` from current directory
- ▶ `rm -r mydir`: Remove directory `mydir` and all its contents
- ▶ **`rm` does not move files to the recycle bin! It deletes them permanently from memory!**

MiniTask

Create a new file into your temporary directory.

1. Create a new file called `my_testfile` with `touch`
2. Execute `ls -l`. Note the details
3. Add text to the file. Try the command: `echo 'write some text here' >> my_testfile`
4. Read the file contents with: `cat my_testfile`
5. Try `echo 'some other text' >> my_testfile`. Read the contents. Now use only a single `>`. What happens to the file?
6. Remove the file with `rm -i my_testfile`. Can you remove the file? Check what the `-i` flag does by invoking `man rm`

Flags, help and quit

Most commands have flags which allow you to ask for more nuanced execution. For example

```
ls -l
```

The flags are too numerous to cover. However, you can get help on commands by writing *man command* or adding the `--help` flag, for example:

```
man ls  
ls --help
```

You can close the terminal by executing *exit*. You can stop commands (or executions of commands) in both the terminal and Python by hitting *Control + C*.

File I/O

For the next three slides we'll use some more advanced commands and play around with a larger text file. The file is an open version of the book "Alice in Wonderland" by Lewis Carroll, which you can download [here](#).

For convenience, you can place the file in the folder you just created.

Searching text with grep

grep (Global Regular Expression Print) searches for specific patterns within a file or input stream.

```
grep "pattern" filename
```

- ▶ **-i: Ignore case** (searches for "alice" and "Alice")
- ▶ **-v: Invert match** (shows lines that *do not* contain the pattern)
- ▶ **-n: Line numbers** (shows which line the match is on)
- ▶ **-c: Count** (gives the total number of matching lines)

Example: Find all mentions of "Rabbit" regardless of capitalization:

```
grep -i "rabbit" Alice_in_Wonderland.txt
```

Piping: Connecting Commands

The pipe operator `|` allows you to use the **output** of one command as the **input** for another.

```
command1 | command2 | command3
```

With pipes, instead of saving a file and opening it in another program, you can "stream" data through a chain of tools to filter and transform it instantly.

Example: See only the first 5 lines containing the word "Queen":

```
grep "Queen" Alice_in_Wonderland.txt | head -n 5
```

The Stream Editor: sed

sed is used for word and pattern replacement. It processes text line-by-line.

```
echo "I love apples" | sed 's/apples/oranges/'
```

- ▶ s: Stands for "substitute"
- ▶ /old/new/: The first word is what to find, the second is the replacement.

By default, sed only replaces the *first* match on each line. Use g to replace every instance.

```
echo "cat cat cat" | sed 's/cat/dog/'  
echo "cat cat cat" | sed 's/cat/dog/g'
```

MiniTask

Execute `grep`, `head`, `sed` and `diff` on the file `Alice_in_Wonderland.txt` to achieve the following:

1. Use `grep` to find all lines that *do not* contain the word "Rabbit", then pipe the output to `head -n 10` to see only the first 10 results.
2. Find all lines containing "Alice" and use `sed` to replace the word "Alice" with "Heidy". Save the outcome in a new file called `Heidy_in_Wonderland.txt`
3. Check out the `diff` command by running

```
diff 'Alice_in_Wonderland.txt' 'Heidy_in_Wonderland.txt'
```

Other useful commands

You can do much more with the command line, which we will not cover extensively here, but feel free to check it out:

- ▶ Compress files with `zip` and `tar`
- ▶ Secure file transfer and remote connection with `scp`, `ssh`
- ▶ Display system information/processes with `htop`, `df`
- ▶ Get super user access with `su` and `sudo`
- ▶ Other build in languages like `awk`

including many more that you may encounter. The procedure for using them is the same.

Outline

Introduction

Research Oriented Programming

Your project

The command line interface

Summary

A look ahead

In this class, we'll focus on

- ▶ Learning Git and Python
- ▶ Structuring and carrying out a team project
- ▶ Leveraging AI assistance in our workflows, with a continuous focus on learning.

No exam, only project and take-home assignments (binary grading). Un-enroll **before** week 3 if you will not commit to the project. Choose or define a project that meets criteria.

A note of caution: Cognitive load and genAI

SCIENCE & TECH

Is AI dulling our minds?

Experts weigh in on whether tech poses threat to critical thinking, pointing to cautionary tales in use of other cognitive labor tools

Liz Mineo | Harvard Staff Writer

November 13, 2025 • long read

A note of caution: Cognitive load and genAI

SCIENCE & TECH

Is AI dulling our minds?

Experts weigh in on whether tech poses threat to critical thinking, pointing to cautionary tales in use of other cognitive labor tools

Liz Mineo | Harvard Staff Writer

November 13, 2025 • long read

The Impact of Generative AI on Critical Thinking: Self-Reported Reductions in Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers

Hao-Ping (Hank) Lee
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA
haopingl@cs.cmu.edu

Ian Drosos
Microsoft Research
Cambridge, United Kingdom
t-iandrosos@microsoft.com

Advait Sarkar
Microsoft Research
Cambridge, United Kingdom
advait@microsoft.com

Sean Rintel
Microsoft Research
Cambridge, United Kingdom
serintel@microsoft.com

Nicholas Wilson
Microsoft Research
Cambridge, United Kingdom
niwilson@microsoft.com

Lev Tankelevitch
Microsoft Research
Cambridge, United Kingdom
levt@microsoft.com

Richard Banks
Microsoft Research Cambridge
Cambridge, United Kingdom
rbanks@microsoft.com

A note of caution: Cognitive load and genAI

SCIENCE & TECH

Is AI dulling our minds?

Experts weigh in on whether tech poses critical thinking, pointing to cautionary tales of other cognitive labor tools

Liz Mineo | Harvard Staff Writer

November 13, 2025 • long read

Research Economic Futures Commitments ▾ Learn ▾ News Try Claude ▾

Alignment

How AI assistance impacts the formation of coding skills

Jan 29, 2026

The Impact of Generative AI on Critical Thinking: Self-Reported Reductions in Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers

Hao-Ping (Hank) Lee
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA
haopingl@cs.cmu.edu

Ian Drosos
Microsoft Research
Cambridge, United Kingdom
t-iandrosos@microsoft.com

Advait Sarkar
Microsoft Research
Cambridge, United Kingdom
advait@microsoft.com

Sean Rintel
Microsoft Research
Cambridge, United Kingdom
serintel@microsoft.com

Nicholas Wilson
Microsoft Research
Cambridge, United Kingdom
niwilson@microsoft.com

Lev Tankelevitch
Microsoft Research
Cambridge, United Kingdom
levt@microsoft.com

Richard Banks
Microsoft Research Cambridge
Cambridge, United Kingdom
rbanks@microsoft.com

A note of caution: Cognitive load and genAI

SCIENCE & TECH

Is AI dulling our minds?

Experts weigh in on whether tech poses critical thinking, pointing to cautionary tales of other cognitive labor tools

Liz Mineo | Harvard Staff Writer

November 13, 2025 • long read

Research Economic Futures Commitments Learn News Try Claude

Alignment

How AI assistance impacts the formation of coding skills

Jan 29, 2026

The Impact of Generative AI on Critical Thinking: Self-Reported Reductions in Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers

[nature](#) > [scientific reports](#) > [articles](#) > [article](#)

Article | [Open access](#) | Published: 29 April 2025

Human-generative AI collaboration enhances task performance but undermines human's intrinsic motivation

[Suqing Wu](#), [Yukun Liu](#) , [Mengqi Buan](#), [Siyu Chen](#) & [Xiao-Yun Xie](#)

[Scientific Reports](#) **15**, Article number: 15105 (2025) | [Cite this article](#)

46k Accesses | **30** Citations | **447** Altmetric | [Metrics](#)

Advait Sarkar
Microsoft Research
Cambridge, United Kingdom
advait@microsoft.com

Sean Rintel
Microsoft Research
Cambridge, United Kingdom
serintel@microsoft.com

Nicholas Wilson
Microsoft Research
Cambridge, United Kingdom
niwilson@microsoft.com

Lev Tankelevitch
Microsoft Research
Cambridge, United Kingdom
levt@microsoft.com

Richard Banks
Microsoft Research Cambridge
Cambridge, United Kingdom
rbanks@microsoft.com

A note of caution: Cognitive load and genAI

SCIENCE & TECH

Is AI dulling our minds?

Experts weigh in on whether tech poses critical thinking, pointing to cautionary tales of other cognitive labor tools

Liz Mineo | Harvard Staff Writer

November 13, 2025 • long read

ANTHROPIC

Research Economic Futures Commitments Learn News Try Claude

Alignment

How AI assistance impacts the formation of coding skills

Jan 29, 2026

BJET British Journal of Educational Technology

BERA

ORIGINAL ARTICLE | [Open Access](#) | [Share](#)

The role of generative AI in collaborative problem-solving of authentic challenges

[Nancy Lauer](#) [Yan Wang](#) [Ming Ma](#) [Zhichun Liu](#) [Leon Lee](#) [Shuhui Feng](#) [Xiao Hu](#) [Jack Tsao](#)

First published: 29 August 2025 | <https://doi.org/10.1111/bjet.70010> | [VIEW METRICS](#)

[nature](#) > [scientific reports](#) > [articles](#) > [article](#)

Article | [Open access](#) | Published: 29 April 2025

Human-generative AI collaboration enhances task performance but undermines human's intrinsic motivation

[Suqing Wu](#) [Yukun Liu](#) [Mengqi Buan](#) [Siya Chen](#) & [Xiao-Yun Xie](#)

[Scientific Reports](#) **15**, Article number: 15105 (2025) | [Cite this article](#)

46k Accesses | **30** Citations | **447** Altmetric | [Metrics](#)

Generative AI on Critical Thinking: Self-Reported Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers

Advait Sarkar
Microsoft Research
Cambridge, United Kingdom
advait@microsoft.com

Sean Rintel
Microsoft Research
Cambridge, United Kingdom
searintel@microsoft.com

Nicholas Wilson
Microsoft Research
Cambridge, United Kingdom
niwilson@microsoft.com

Lev Tankelevitch
Microsoft Research
Cambridge, United Kingdom
levt@microsoft.com

Richard Banks
Microsoft Research Cambridge
Cambridge, United Kingdom
rbanks@microsoft.com

A note of caution: Cognitive load and genAI

SCIENCE & TECH

Is AI dulling our minds?

Experts weigh in on whether tech poses critical thinking, pointing to cautionary tales of other cognitive labor tools

Liz Mineo | Harvard Staff Writer

November 13, 2025 • long read

Research Economic Futures Commitments Learn News Try Claude

Alignment

How AI Impacts Skill Formation and Cognitive Skills

Judy Hanwen Shen* Alex Tamkin†

February 3, 2026

BJET British Journal of Educational Technology

ORIGINAL ARTICLE | Open Access

The role of generative AI in collaborative problem-solving of authentic challenges

Nancy Law, Nan Wang, Ming Ma, Zhichun Liu, Leon Lei, Shuhui Peng, Xiao Hu, Jack Tao

First published: 29 August 2025 | <https://doi.org/10.1111/bjet.70010> | [VIEW METRICS](#)

[nature](#) > [scientific reports](#) > [articles](#) > [article](#)

Article | [Open access](#) | Published: 29 April 2025

Human-generative AI collaboration enhances task performance but undermines human's intrinsic motivation

Suping Wu, Yukun Liu, Mengqi Bu, Siyu Chen & Xiao-Yun Xie

Scientific Reports **15**, Article number: 15105 (2025) | [Cite this article](#)

46k Accesses | 30 Citations | 447 Altmetric | [Metrics](#)

Generative AI on Critical Thinking: Self-Reported Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers

Advait Sarkar
Microsoft Research
Cambridge, United Kingdom
advait@microsoft.com

Lev Tankelevitch
Microsoft Research
Cambridge, United Kingdom
levt@microsoft.com

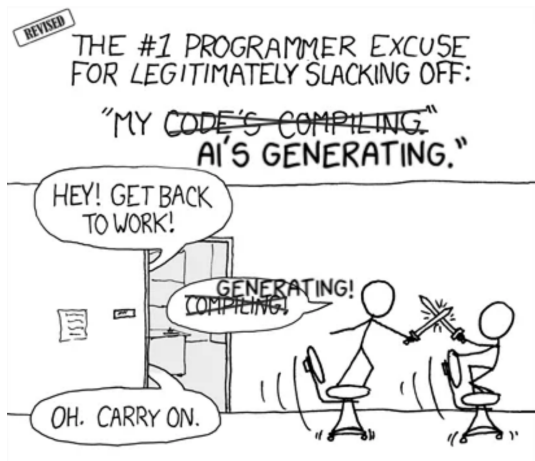
Sean Rintel
Microsoft Research
Cambridge, United Kingdom
serintel@microsoft.com

Nicholas Wilson
Microsoft Research
Cambridge, United Kingdom
niwilson@microsoft.com

Richard Banks
Microsoft Research Cambridge
rbanks@microsoft.com

On generating code

Code is cheap, but good software is not. Especially when it comes to applying domain expertise and working in groups!



Comic edited from [xkcd](#).